ArUco Marker-Based Precision Landing

Vishnu S L (23BRS1122) | Kanishk M (23BRS1029) | Prasanna E (23BRS1001)
Guide: Dr. D. Sathian | School of Computer Science and Engineering | Vellore Institute of Technology, Chennai

1. ABSTRACT

A **vision-based precision-landing system** for a **Holybro X500 V2** quadcopter with a PX4 **Pixhawk 6X** flight controller. A down-facing Raspberry Pi camera detects an **ArUco marker** (DICT_4X4_50, ID 0) with OpenCV, estimates its pose with calibrated *solvePnP*, converts it to the NED frame and streams it to a **Pixhawk 6X** as MAVLink **LANDING_TARGET**. The pilot flies manually; once the marker is confirmed (15 frames, armed, airborne, within 15 m) the script switches PX4 to **AUTO.PRECLAND** (fallback AUTO.LAND) and PX4 centres over the marker, lands and disarms. RC mode and kill switches stay active as a safety net.

2. PROBLEM STATEMENT

- GPS is accurate only to metres, so landing on a small pad is unreliable.
- Manual landing needs skill; a slip in the last metres can damage the drone or payload.
- A printed marker gives a cheap, camera-only, centimetre-level reference, but vision must **never fight the pilot** and must engage only when it is safe.

3. OBJECTIVES

- Calibrate the camera and measure marker pose (X, Y, Z) at 640×480.
- Detect ArUco ID 0 in real time on a Raspberry Pi with a smoothing filter.
- Convert pose to NED using PX4 attitude and send LANDING_TARGET over MAVLink at 15 Hz.
- Engage AUTO.PRECLAND automatically, keeping pilot mode-switch / kill-switch override.

4. SOFTWARE MODULES

Script	Purpose
calibrate_charuco.py	ChArUco camera calibration
aruco_detect.py / aruco_pose.py	Detection and 3-D pose test
pixhawk_test.py	MAVLink heartbeat check
aruco_pixhawk_test.py	Pose + Pixhawk link test
landing_target_test.py	Bench test (disarmed, props off)
aruco_auto_land.py	Full auto-land state machine

5. HARDWARE SETUP



Holybro X500 V2 frame



Pixhawk 6X flight controller



Raspberry Pi with camera module



Landing marker: ArUco ID 0

Component	Role
Holybro X500 V2	Quadcopter frame, motors, ESCs
Pixhawk 6X (PX4)	Flight controller, precision-land logic
Raspberry Pi + camera	Companion computer; down-facing 640×480 @ 30 fps
UART link	Pi ↔ TELEM2, 57600 baud, MAVLink 2
RC transmitter	Manual flight, mode / kill switch
ArUco marker	DICT_4X4_50, ID 0, 16.8 cm (6 cells × 2.8 cm)

6. SYSTEM ARCHITECTURE

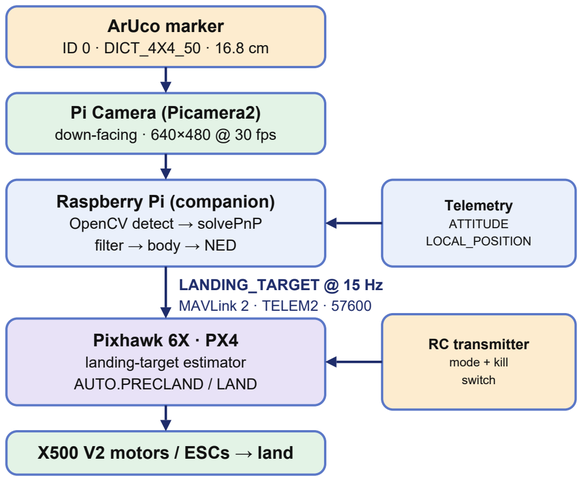


Figure 1: Marker → camera → Raspberry Pi → MAVLink → PX4. PX4 telemetry flows back to the Pi for the frame transform; the pilot's RC link stays connected to the Pixhawk at all times.

7. METHODOLOGY

1. Calibration. *calibrate_charuco.py* captures 20–30 views of a 7×10 ChArUco board (24 mm squares, DICT_5X5_100) and saves the intrinsics and distortion to *camera_calibration.npz*.

2. Detection and pose. *ArucoDetector* with sub-pixel corner refinement finds ID 0. *solvePnP* (**IPPE_SQUARE**) on the four corners of the 16.8 cm marker returns (x, y, z); distance = |t|.

3. Frames and filter. Camera → body: forward = −y, right = +x, down = +z. An exponential filter (α = 0.35) smooths it; the roll-pitch-yaw rotation maps it to NED and *LOCAL_POSITION_NED* is added.

4. Engage. Every frame the state machine checks the safety gates and, when all pass, sends *MAV_CMD_DO_SET_MODE* to enter AUTO.PRECLAND.

8. LANDING STATE MACHINE

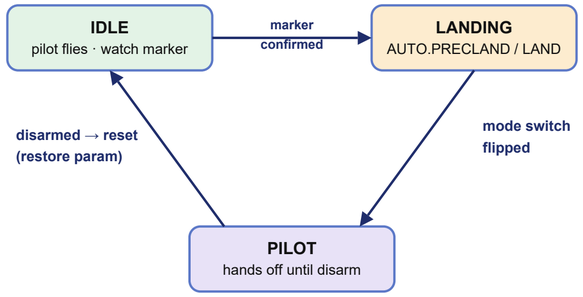


Figure 2: IDLE → LANDING on a confirmed marker; a pilot mode-switch change moves it to PILOT and the script stays hands-off until the vehicle disarms.

9. CALIBRATION & PARAMETERS

Item	Value
Image size	640 × 480 px
Focal length fx, fy	649.7, 651.6 px
Principal point cx, cy	325.8, 262.5 px
Reprojection RMS error	0.68 px
Pose solver	IPPE_SQUARE
LANDING_TARGET rate	15 Hz
Filter factor α	0.35

Table 1: Output of *calibrate_charuco.py* and key runtime parameters of *aruco_auto_land.py*.

10. RESULTS

```
Searching for ArUco ID 0...
Searching for ArUco ID 0...
Searching for ArUco ID 0...
Searching for ArUco ID 0...
Searching for ArUco ID 0...
Searching for ArUco ID 0...
BODY    | F=+0.024 R=-0.018 D=+0.612
REL NED| N=-0.024 E=+0.003 D=+0.612
VEH NED| N=-15.091 E=+1.963 D=-3.256
TARGET | N=-15.116 E=+1.966 D=-2.643
LANDING_TARGET sent: 141
```

Figure 3: Live output of *landing_target_test.py* (bench test).

- **Marker detected** at about 0.61 m, only 2.4 cm forward and 1.8 cm left of the camera centre.
- **Frame chain verified:** target D = vehicle D + relative D (−3.256 + 0.612 = −2.643 m).
- **LANDING_TARGET streamed** to PX4: 141 messages sent in this run.
- **“Searching for ArUco ID 0”** is shown, and nothing is sent, while the marker is out of view.
- **Staged testing:** bench (props off), then tethered, then open field.

11. CONCLUSION & FUTURE WORK

A low-cost camera and a printed marker become an automatic landing aid for a PX4 drone: calibration, pose estimation, MAVLink streaming and a guarded mode switch work together while the pilot stays in control.

Future work

- Moving-platform landing with motion prediction.
- Nested markers visible down to touchdown; wind and low-light tests.
- Logged flight metrics: landing error and success rate.

12. REFERENCES

1. S. Garrido-Jurado et al., “Automatic generation and detection of highly reliable fiducial markers under occlusion,” *Pattern Recognition*, 47(6), 2014.
2. A. Paris, B. T. Lopez, J. P. How, “Dynamic Landing of an Autonomous Quadrotor on a Moving Platform in Turbulent Wind Conditions,” arXiv:1909.11071.
3. OpenCV ArUco detection: docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html
4. PX4 Precision Landing: docs.px4.io/main/en/advanced_features/precision_landing.html
5. MAVLink LANDING_TARGET: mavlink.io/en/messages/common.html